

JavaScript – DOM – funkcje

Spis treści

JavaScript – DOM – funkcje	1
Tworzenie elementów DOM	2
createElement()	2
Dodawanie elementów DOM do strony	2
appendChild()	2
Poszukiwania rodziny (rodzica, braci)	2
previousElementSibling	2
nextElementSibling	3
parentElement	4

Tworzenie elementów DOM

createElement()

```
let div = document.createElement("div")
```

Dzięki niej stworzymy pusty kontener, który przechowywany jest **tymczasowo w pamięci JavaScript**, a nie od razu w drzewie DOM, dlatego też nie zobaczymy go na stronie, aby to zrobić musimy go dodać za pomocą **appendChild()**

Dodawanie elementów DOM do strony

appendChild()

Jeżeli potrzebujemy dodać element do strony HTML po jej wyświetleniu to musimy najpierw skorzystać z metody **appendChild()**, np. dodać:

- na końcu body za pomocą właściwości body (linia nr 1)
- do konkretnego elementu znajdującego się na stronie (linia nr 3)

```
1      document.body.appendChild(div)
2
3      document.querySelector("div").appendChild(div)
```

Poszukiwania rodziny (rodzica, braci)

Nie raz będziemy potrzebować odszukać element, który znajduje się obok jakiegoś elementu. Dla przykładu nasz guzik ma przekreślić przypisany do siebie tekst w paragrafie

```
<div>
  <p>Paragraph</p>
  <button>Done</button>
</div>
```

previousElementSibling

Jak zatem dla powtarzających się elementów w strukturze HTML odnaleźć ten, który jest tuż obok klikniętego guzika? Możemy skorzystać ze struktury DOM oraz ich funkcjom, zobacz:

```
<script>
document.querySelector("button").addEventListener("click", (event)=>{
  let p = event.target.previousElementSibling
  p.style = "text-decoration:line-through"
})
</script>
```

Za pomocą **previousElementSibling** dostaliśmy dostęp do znacznika poprzedzającego nasz guzik (odnaleźliśmy go).

W przypadku funkcji, która uruchamiana jest po kliknięciu w guzik, wykorzystujemy zmienną event, która przechowuje informacje o zdarzeniu.

nextElementSibling

Podobnie za pomocą **nextElementSibling** uzyskamy dostęp do elementu, który jest za wskazanym.

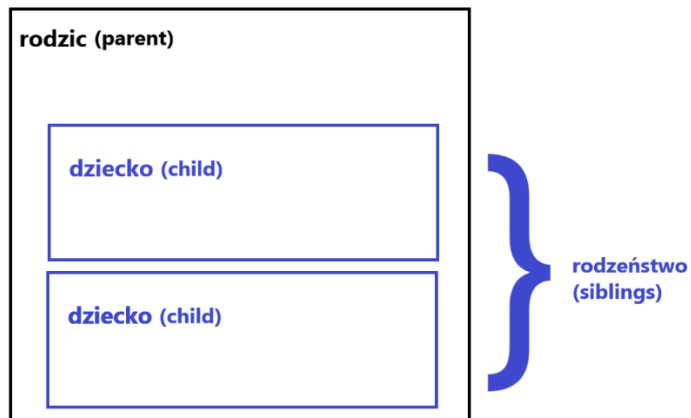
```
<div>
  <p>Paragraph</p>
  <button>Done</button>
</div>
```

Tym razem chcemy zmienić kolor guzika za pierwszym paragrafem na czerwony. Kod JavaScript może wyglądać tak:

```
const button = document.querySelector("p").nextElementSibling
button.style = "color: red"
```

parentElement

Mianem rodzica (parent) określamy element nadrzędny (element wyżej lub po prostu ten element, w którym znajduje się jakiś element).



```
<div>
  <p>Paragraph</p>
  <button>Done</button>
</div>
```

Dla powyższego schematu HTML **rodzicem** jest `<div>`, a **dziećmi** `<p>` oraz `<button>`. Natomiast dla siebie nawzajem `<p>` oraz `<button>` są **rodzeństwem**.

Jeżeli chcielibyśmy wyświetlić rodzica lub uzyskać dostęp do jakiegoś elementu użyjemy poniższych właściwości **parentElement** oraz **nodeName**

```
document.querySelector("button").addEventListener("click", (event)=>{  
  let div = event.target.parentElement  
  console.log(div.nodeName)  
})
```

Trzeba w tym miejscu zaznaczyć, że powyższy skrypt działa w następujący sposób:

- klikamy w przycisk na stronie
- przekazujemy do funkcji obsługującej zdarzenie kliknięcia argument jakim jest zmienna przechowująca zdarzenie (event)
- tworzymy zmienną div i przypisujemy do niej rodzica elementu, na którym zostało wykonane zdarzenie
- wyświetlamy w konsoli nazwę rodzica (nazwę elementu)

Wynik

jego

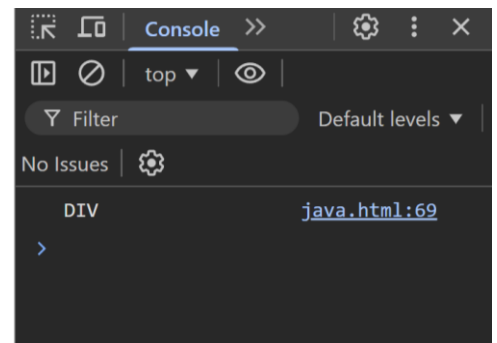
działania

widzimy

poniżej:

Paragraph

Done



Powyższy kod sprawi, że tekst guzika za paragrafem zmieni kolor na czerwony.

Paragraph

Done

1.